

Cheatsheet

CSR-patroon, Repository en PHP-klassen

Strict types

Zet dit bovenaan elk PHP-bestand. PHP dwingt dan de opgegeven types af: je kunt geen `string` doorgeven waar een `int` verwacht wordt. Zonder strict types converteert PHP types stilletjes, wat tot verrassende fouten kan leiden.

```
<?php
declare(strict_types=1);
```

Klassen en properties

Een **property** is een variabele die bij een object hoort. Je declareert hem in de klasse-body met een access modifier en een type.

```
class User
{
    public int $id;
    public string $username;
    public string $email;
}
```

Access modifiers

Modifier	Zichtbaar voor
public	Iedereen
protected	De klasse zelf en subklassen
private	Alleen de klasse zelf
readonly	Eenmalig schrijfbaar, daarna alleen lezen

Gebruik `private` voor alles wat een intern implementatiedetail is. Maak properties alleen `public` als andere code er echt bij moet.

Constructor

De constructor wordt automatisch aangeroepen bij `new KlasseNaam(...)`. Gebruik hem om het object in te stellen met de waardes die het nodig heeft.

```
class User
{
    private int $id;
    private string $username;

    public function __construct(int $id, string $username)
    {
        $this->id = $id;
        $this->username = $username;
    }
}

$user = new User(1, 'jan');
```

Constructor property promotion

Kortere schrijfwijze: declareer en wijs toe in één stap door de access modifier direct in de constructor-parameter te schrijven. PHP doet de rest.

```
class User
{
    public function __construct(
        private readonly int $id,
        private readonly string $username,
    ) {}
}
```

Hetzelfde resultaat als hierboven, maar zonder de herhalingen. Dit is de stijl die we in dit vak gebruiken.

Return types

```
public function getUsername(): string { ... } // altijd een string
public function findUser(): ?User { ... } // User of null
public function save(): void { ... } // geeft niets terug
```

?Type (nullable type) betekent: de methode geeft dit type terug, of `null`. Gebruik dit als het normaal is dat iets niet gevonden wordt.

Het CSR-patroon

Controller → Service → Repository → Database

Laag

Verantwoordelijkheid

Controller	HTTP-input ontvangen, response terugsturen
Service	Businesslogica: mag dit? Klopt dit?
Repository	Data lezen en schrijven naar de database

Gouden regel: elke laag praat alleen met zijn directe buur. De controller roept de service aan, de service roept de repository aan. De controller schrijft nooit SQL.

Repository

Een repository is de enige klasse die SQL schrijft voor een bepaald model. De rest van de applicatie vraagt gewoon om objecten en weet niet hoe die uit de database komen.

```
class UserRepository
{
    public function __construct(private readonly PDO $pdo) {}

    public function findByUsername(string $username): ?User
    {
        $stmt = $this->pdo->prepare('SELECT * FROM users WHERE username = ?');
        $stmt->execute([$username]);
        $row = $stmt->fetch();

        return $row ? $this->hydrate($row) : null;
    }

    private function hydrate(array $row): User
    {
        return new User(
            id: (int) $row['id'],
            username: $row['username'],
            email: $row['email'],
        );
    }
}
```

De PDO-verbinding is `private`: andere klassen hoeven niet te weten hoe de repository intern werkt.

Hydrate

`hydrate()` zet een ruwe database-rij om naar een object van de bijbehorende klasse.

Zonder hydrate werk je overal in de applicatie met een associatieve array:

```
$row = $stmt->fetch();
echo $row['username']; // je moet de kolomnaam kennen
```

Met hydrate werk je met een object. Properties lees je met de **pijl-operator** (`->`):

```
$user = $this->hydrate($row);  
echo $user->username; // pijl-operator, IDE helpt je
```

Stel dat je op je registratiepagina, profielpagina en adminpagina allemaal met gebruikersdata werkt. Zonder hydrate moet elke pagina de kolomnamen uit de database kennen:

`$row['username']`, `$row['email']` enzovoort. Verandert een kolomnaam in de database? Dan moet je dat op elke losse pagina opzoeken en aanpassen.

Met hydrate stel je de kolomnamen eenmalig in, in de `hydrate()`-methode zelf. Daarna werken alle pagina's met een `User`-object en hoeven ze niets van de databasetabel te weten. Alleen `hydrate()` kent de tabel.

- Properties lees je met de pijl-operator: `$user->username`
- Je IDE geeft autocompletion op properties, niet op losse array-keys
- Kolomnaam verandert? Pas het alleen aan in `hydrate()`, nergens anders

Dependency injection via de constructor

Elke laag ontvangt zijn afhankelijkheden via de constructor. In `index.php` bouw je de lagen op van binnen naar buiten:

```
$pdo = new PDO('mysql:host=localhost;dbname=mijn_db', 'root', '');  
$userRepository = new UserRepository($pdo);  
$authService = new AuthService($userRepository);  
$authController = new AuthController($authService);
```

Zo is elke klasse op zichzelf te begrijpen en te vervangen. De `UserRepository` weet niet waar zijn PDO vandaan komt. Dat is de kracht van dit patroon.